MODEL AGNOSTIC AI OS

AUTOMATED INFRASTRUCTURE

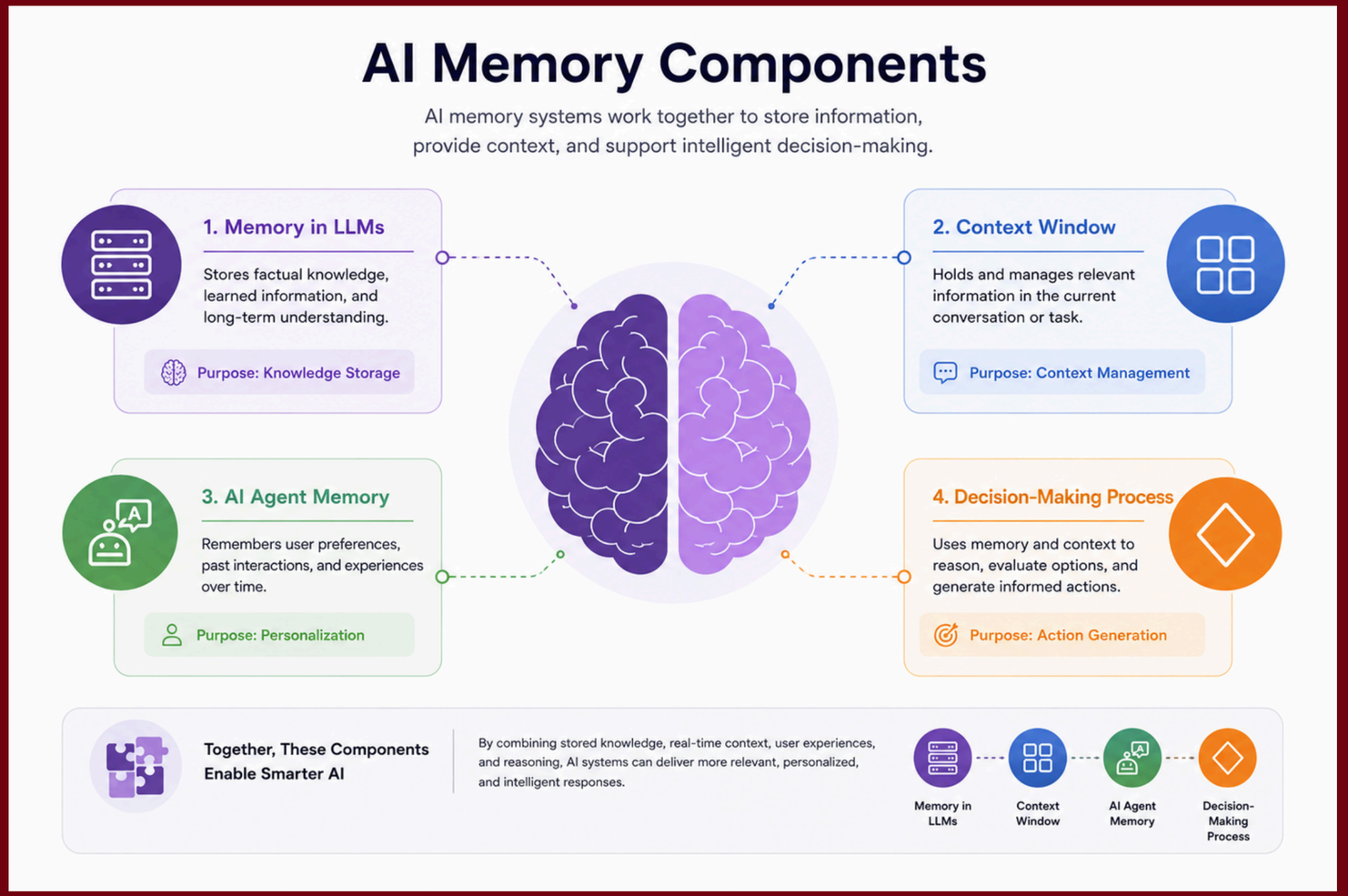
Designed & Architected by
Karan Paliwal

A model-agnostic Obsidian OS that helps AI agents remember project truth, session history, corrections, and durable lessons without loading everything.

AI AGENTS DON'T REALLY REMEMBER.

Most AI work today lives inside long chats. A decision gets made, a bug gets fixed, a pattern is learned, and then the next session starts from zero.
The user becomes the memory layer, constantly re-explaining the same project, the same constraints, and the same mistakes.

Chat → Lost context → Re-explain → Repeat

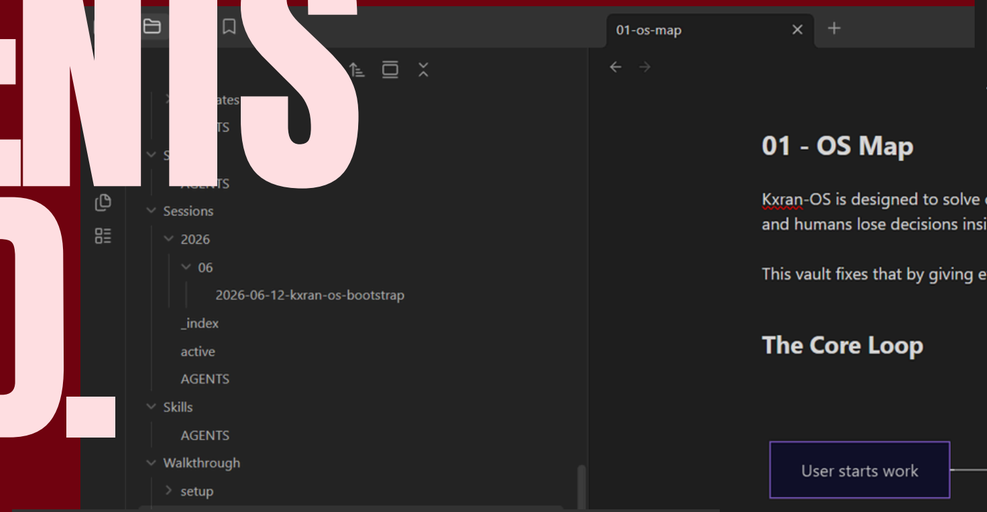


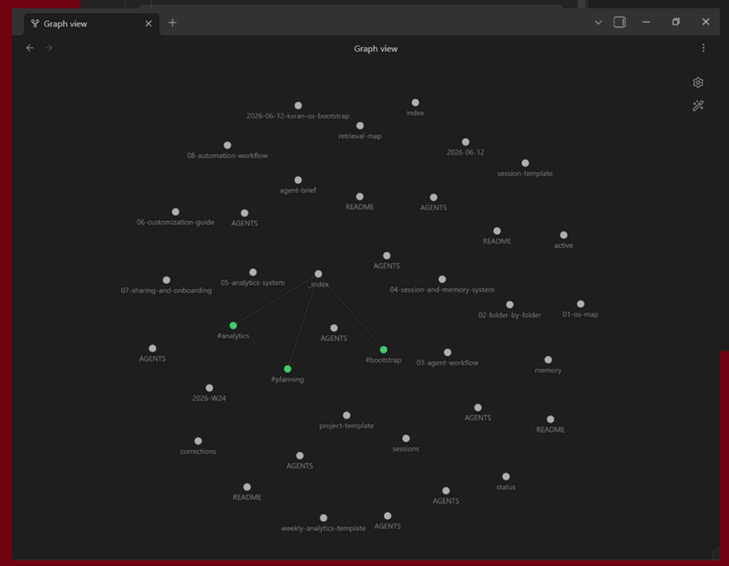
The issue is not intelligence. It is continuity.

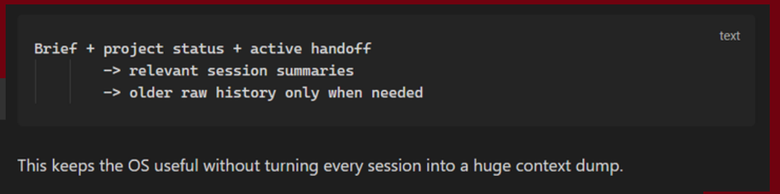
THE OS: A VAULT THAT AGENTS CAN READ.

AI-OS turns an Obsidian vault into a shared filesystem memory for AI-agent work.
Instead of loading every old conversation, the agent reads the current brief, project status, active handoff, and only the specific history needed for the task.

Agent Brief:
/Retrieval Map
/Project Status
/Active Session
/Relevant History Only







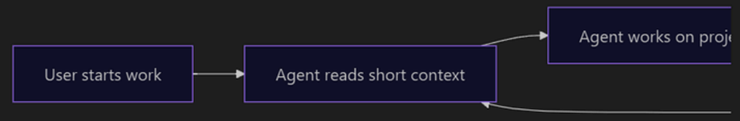
This keeps the OS useful without turning every session into a huge context dump.

01 - OS Map

Kxran-OS is designed to solve one recurring problem: AI agents forget context between sessions, and humans lose decisions inside long chats.

This vault fixes that by giving every future session a shared filesystem memory.

The Core Loop

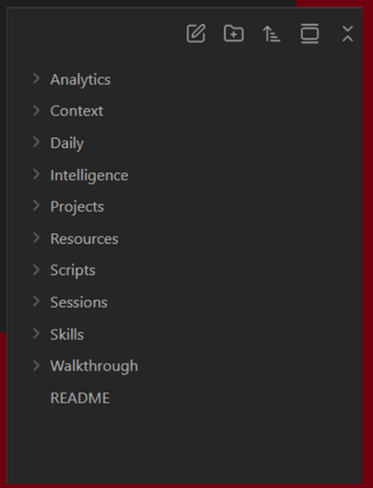


Why It Is Token Efficient

The OS does not ask the agent to read everything.

Instead, the agent reads:

1. the short brief;
2. the current project status;
3. the active session handoff;
4. only the relevant older sessions if a specific question requires them.



The system is not a chat archive. It is a retrieval layer.

FOUR MEMORY TYPES. ONE SOURCE OF TRUTH.

The Four Memory Types

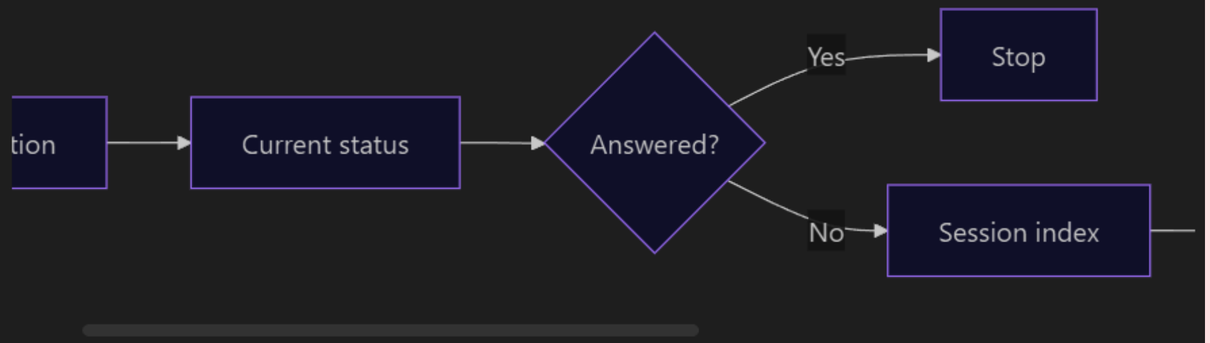
Kxran-OS separates memory into four types.

Type	Stored In	Purpose
Current state	`Projects/<project>/status.md`	What is true now
Durable memory	`Context/memory.md`	Facts and preferences that persist
Corrections	`Context/corrections.md`	Mistakes and behavior chang
History	`Sessions/YYYY/MM/`	What happened in past sessi

AI-OS separates memory so future agents know what to trust. Current state is not mixed with old history. Corrections are not buried inside session notes. Durable lessons are promoted into memory only when they should persist.

Why this matters:
/Status tells the agent what is true now
/Memory keeps reusable facts and preferences
/Corrections prevent repeated mistakes
/Sessions preserve history without polluting startup context

Session Retrieval



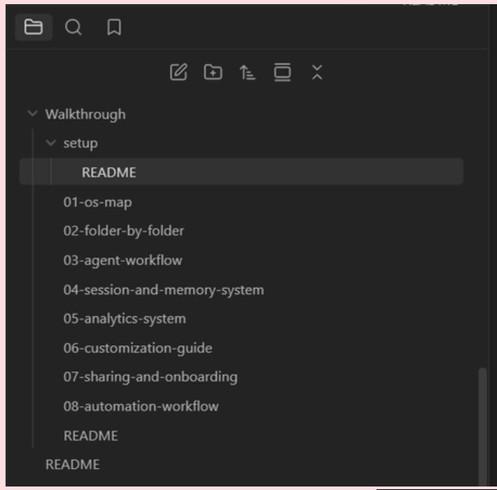
The retrieval rule is simple: search before opening, summarize before promoting, and promote only what should persist.

Consistency comes from structure, not longer prompts.

FROM SESSION START TO SESSION MEMORY.

The workflow is designed for real project work. Start a session, let the agent read the right context, work normally, then close the session into the OS.

Register project once:
/Start session with a goal
/Agent reads short context
/Work happens
/Closeout saves summary
/Memory/corrections get promoted
/Weekly analytics reviews the system



How The Project Appears In Obsidian

After registration, open Obsidian and go to:

```
Projects/project-name/
```

You will see:

- `README.md` : what the project is;
- `status.md` : current truth;
- `sessions.md` : links to relevant sessions.

As you work, the graph grows because sessions, project files, memory, cc link together.

Back up:

```
& "C:\Codex Shit\Kxran-OS\Scripts\Backup-KxranOS.cmd" -Message "Update OS memory"
```
PowerShell

Setup - Link A New Project To Kxran-OS

Use this guide whenever you start a new project and want Kxran-OS to remember it.

The goal is simple:

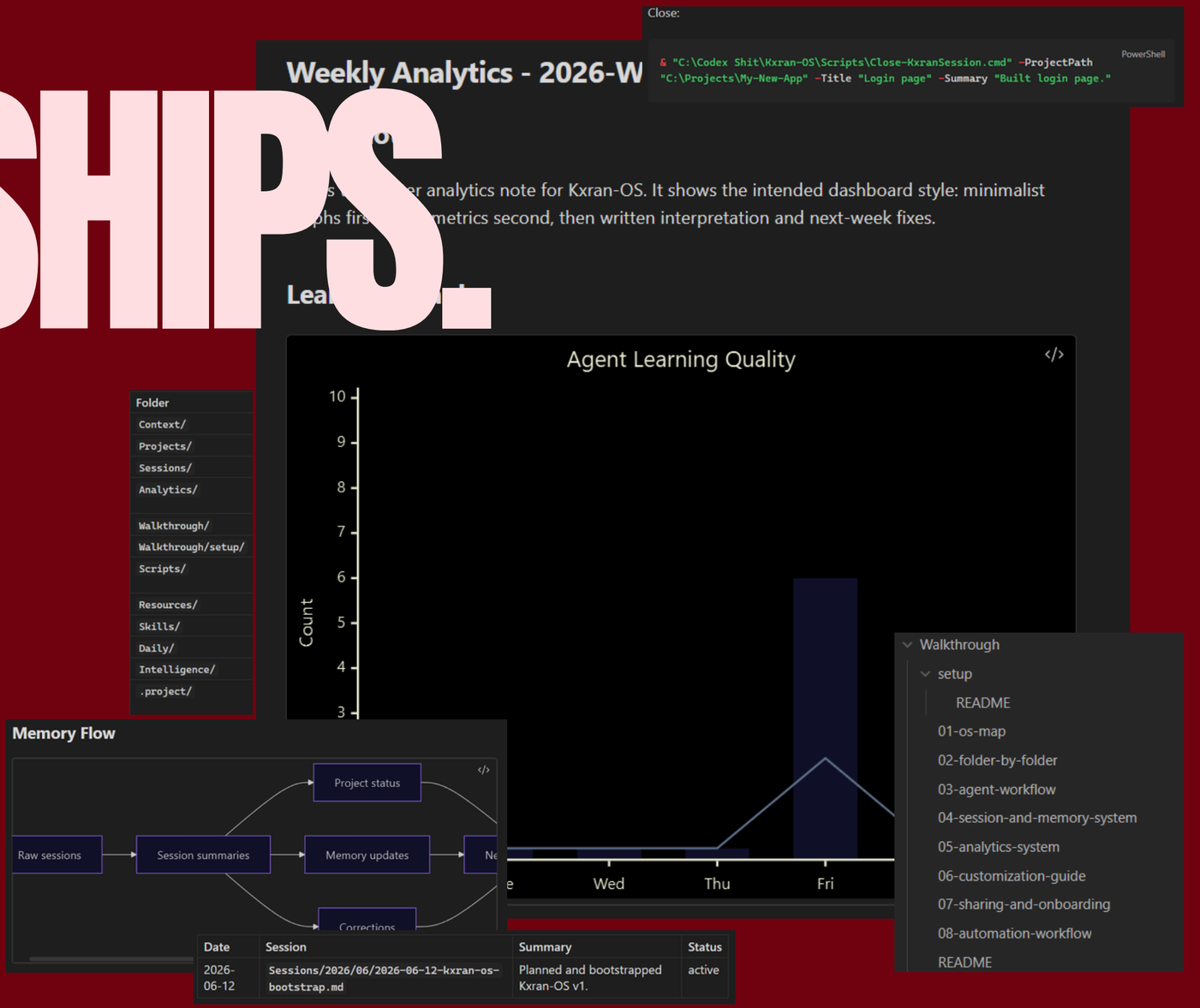
1. Register the project once.
2. Start a Kxran-OS session when you begin work.
3. Close the session when meaningful work is done.
4. Back up Kxran-OS when you want the memory saved to GitHub.

The closeout is where raw work becomes reusable memory.

WHAT V1 ACTUALLY SHIPS.

V1 is intentionally simple. No custom plugin. No MCP automation. No forced sync. The goal was to prove the local workflow before adding heavier automation.

/Obsidian vault structure
/Project truth files
/Agent brief and retrieval map
/Chronological session archive
/Active session handoff
/Durable memory and corrections
/Weekly analytics template
/Registration, start, closeout, and backup scripts
/New-project setup walkthrough



Markdown stays the source of truth.

THANK YOU

THANK YOU

AUTOMATED INFRASTRUCTURE

Designed & Architected by
Karan Paliwal

Most agent memory problems are not solved by pasting more context.
They are solved by giving the agent a system for knowing what to read, what to trust, and what to remember.